

Chapter 15 : Probabilistic Reasoning over Time

1. Why Reason Over Time?

- Many real-world problems involve uncertain, changing environments.
- An agent must maintain a belief about how the world evolves over time.
- Actions have long-term consequences and information arrives gradually through noisy observations.
- Examples: robot localization, tracking objects, medical monitoring, weather prediction, planning under uncertainty.

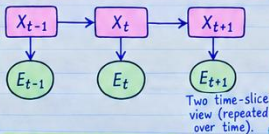
Key Idea: We represent the world as a sequence of time-indexed random variables and use probabilistic models to update beliefs as time passes.

2. Time Slices and State Variables

- We represent the world using discrete time slices $t=0, 1, 2, \dots$
- At each time slice t , there is a set of state variables X_t describing the world.
- We may also have evidence (observation) variables E_t representing sensor readings.
- Capital letters denote random variables; X_t is the set of state variables at time t .

Example: Robot localization

- X_t : robot's location at time t
- E_t : noisy sensor reading of location



3. Transition Model $P(X_t | X_{t-1})$

- Specifies how the state evolves from time $t-1$ to t .
- Encodes the dynamics of the world and the effect of actions (if any).
- Gives the distribution over next state conditional on current state.

Transition model: = probability of next state $P(X_t | X_{t-1})$ given current state.

Example: Robot motion (1D)

- If robot moves Right: with prob. 0.8 it moves to $x+1$, and with prob. 0.2 it stays in x .

$X_{t-1} = x$	$P(X_t = x')$
$x' = x+1$	0.8
$x' = x$	0.2

4. Sensor / Observation Model $P(E_t | X_t)$

- Specifies how likely an observation is given the current state.
- Models the uncertainty and noise in sensors.
- Allows the agent to update its belief about the state after receiving evidence.

Sensor model: = probability of evidence $P(E_t | X_t)$ given the current state.

Example: Noisy location sensor

- If robot is at location x , sensor reports x with prob. 0.7, and an adjacent location with prob. 0.15 each.

True location X_t	$P(E_t = e)$
$e = x$	0.7
$e = x-1$	0.15
$e = x+1$	0.15

5. Markov Assumption

- The next state depends only on the current state, not on the full history.
- Past states are irrelevant given the current state.

Markov assumption:

$$P(X_t | X_{0:t-1}) = P(X_t | X_{t-1})$$

- Similarly, the observation at time t depends only on the current state:

$$P(E_t | X_{0:t}, E_{1:t-1}) = P(E_t | X_t)$$

Intuition:

- The current state X_t is a sufficient statistic of the past for predicting the future.
- Makes inference and learning tractable.

6. First-Order Markov Process

- A sequence of random variables $\{X_0, X_1, X_2, \dots\}$ such that

$$P(X_t | X_{0:t-1}) = P(X_t | X_{t-1}) \quad (\text{for all } t)$$

- Completely specified by an initial distribution and a transition model.
- In the absence of observations, this is a (discrete-time) Markov chain.

Joint distribution:

$$P(X_{0:T}) = P(X_0) \prod_{t=1}^T P(X_t | X_{t-1})$$

7. Stationary Process (Time-Invariant)

- Transition model does not change over time:

$$P(X_t | X_{t-1}) = P(X | X') \quad (\text{same for all } t)$$

- Often a reasonable assumption in many domains (e.g., robot motion, weather models).

8. Hidden Markov Models (HMMs)

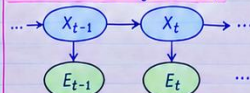
- A simple model for temporal probabilistic reasoning with hidden states.
- States X_t are not directly observable (hidden), but we get noisy observations E_t .
- Consists of:
 - Initial distribution $P(X_0)$
 - Transition model $P(X_t | X_{t-1})$
 - Sensor model $P(E_t | X_t)$
- Typical tasks: filtering (state tracking), prediction, smoothing, and most probable state sequence (e.g., Viterbi).

Note: HMM is a special case of a dynamic Bayesian network with one state variable per time slice.

9. Dynamic Bayesian Networks (DBNs)

- A generalization of HMMs to multiple state and evidence variables.
- Represents the probabilistic dependencies within a time slice and across time slices.
- Consists of a two-time-slice Bayesian network that is repeated over time.

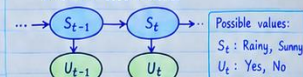
Two time-slice DBN (general form):



- Edges can exist between multiple variables.
- HMM is a DBN with one state variable and one evidence variable per time slice.

10. Example: Umbrella World (Weather)

- Latent state: weather $S_t \in \{\text{Rainy, Sunny}\}$
- Evidence: observer sees umbrella $U_t \in \{\text{Yes, No}\}$
- Model:
 - Transition $P(S_t | S_{t-1})$ captures how weather changes over time.
 - Sensor model $P(U_t | S_t)$ captures how likely an umbrella is seen given the weather.
- We cannot observe the weather directly, but we can infer it from a sequence of umbrella observations.



Goal: Compute $P(S_t | U_{1:t})$ (belief over weather given observations).

"Probabilistic models over time allow an agent to make sense of noisy, incomplete information and to reason about how the world evolves."

1. State Estimation / Filtering

- Filtering (or monitoring) is the problem of estimating the current state of a dynamic system given all evidence observed so far.
- We cannot observe the true state directly, but we receive noisy observations.
- We maintain a probability distribution over the current state based on the history of actions and observations.
- Used in tracking, localization, robot perception, medical monitoring, etc.

Key Idea: Filtering computes the belief state $P(X_t | e_{1:t})$, the probability distribution over the current state given all past evidence up to time t .

2. Belief State $B(X_t)$

- The belief state is a probability distribution over the possible current states.
- It summarizes all relevant information from the past for decision making.
- For discrete state space:

$$B(X_t) = P(X_t | e_{1:t})$$

- The belief state is a sufficient statistic of the history (past states and evidence).
- We do not need to remember the entire history, only the current belief state.

3. Recursive Filtering Equation

- We can compute the new belief state recursively using two steps:

- Prediction:** Predict next state using the transition model.
- Correction:** Update using the new evidence (observation).

Filtering update:

$$P(X_t | e_{1:t}) = \alpha \underbrace{P(e_t | X_t)}_{\text{Prediction (step 1)}} \underbrace{P(X_t | e_{1:t-1})}_{\text{Correction (step 2)}}$$

- First predict the next state distribution using the transition model.
- Then incorporate the new evidence using the sensor model and normalize (α).

Intuition: The belief at time t depends only on the previous belief and the new evidence, not on the entire past.

4. Prediction $P(X_{t+1} | e_{1:t})$

- Often we want to predict the state at the next time step before seeing new evidence.
- This is the one-step-ahead prediction using only the transition model:

$$P(X_{t+1} | e_{1:t}) = \sum_{x_t} \frac{P(X_{t+1} | x_t) P(x_t | e_{1:t})}{P(x_t | e_{1:t})}$$

- We marginalize over the current belief state.
- This gives the prior for time $t+1$, which is then updated when observation e_{t+1} arrives.

5. Umbrella World Filtering Example

- Umbrella world: a simple example with two states: Rain (R) or No Rain ($\neg R$).
- We observe whether the umbrella is seen (U) or not ($\neg U$).
- We compute the belief state $P(R_t | e_{1:t})$ over time using the filtering update.

Models:

- Transition model (weather):

$P(R_{t+1} R_t)$	$R_{t+1} = R$	$R_{t+1} = \neg R$
$R_t = R$	0.7	0.3
$R_t = \neg R$	0.3	0.7

- Sensor model (umbrella observation):

$P(e_t R_t)$	$e_t = U$	$e_t = \neg U$
$R_t = R$	0.9	0.1
$R_t = \neg R$	0.2	0.8

Example: Filtering for two time steps

- Initial belief: $P(R_0) = 0.5$, $P(\neg R_0) = 0.5$
- Observation at $t = 1$: $e_1 = U$

1 Prediction:

$$P(R_1) = 0.7(0.5) + 0.3(0.5) = 0.5$$

$$P(\neg R_1) = 0.3(0.5) + 0.7(0.5) = 0.5$$

2 Correction using $e_1 = U$:

$$P(R_1 | U) \propto 0.9(0.5) = 0.45$$

$$P(\neg R_1 | U) \propto 0.2(0.5) = 0.10$$

- Normalize ($\alpha = 1/0.55$):

$$P(R_1 | U) = 0.45/0.55 = 0.82$$

$$P(\neg R_1 | U) = 0.10/0.55 = 0.18$$

Time step	$P(R_t)$	$P(\neg R_t)$	Observation
$t = 0$	0.50	0.50	—
$t = 1$	0.82	0.18	U

9. Benefits of Filtering

- Maintains an up-to-date estimate of the current state.
- Requires only the current belief state (constant memory).
- Allows real-time monitoring and decision making.
- Works well with noisy and uncertain sensors and actions.
- Widely used in robotics, tracking, and prediction tasks.

6. Forward Algorithm Intuition

- Filtering is also called the forward algorithm in HMM/DBN literature.
- We propagate a forward message (belief state) from left to right in time.
- At each time step, we combine information from the previous belief and the new evidence.
- This is efficient and requires only the current belief state.

Forward message:

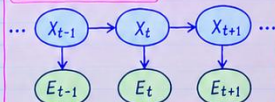
$$\alpha_t(X_t) = P(X_t = x_t | e_{1:t})$$

- Compute α_{t+1} from α_t using prediction and correction.
- Only need to store the current message.

7. Robot Localization Interpretation

- In robot localization, X_t is the robot's location (e.g., a grid cell).
- Actions (movements) are uncertain due to motion noise.
- Sensors (e.g., distance readings) are noisy.
- We maintain a belief distribution over all possible locations and update it as the robot moves and receives observations.

DBN for localization:



- X_t : robot's location at time t .
- E_t : noisy sensor reading (e.g., distance or landmark observation).
- We use the same filtering update to track the robot's current location.

8. Exact vs. Recursive Computation

- Exact computation from the full history:

$$P(X_t | e_{1:t}) = \alpha P(e_{1:t}, X_t)$$

- This requires considering all past evidence and is computationally expensive.
- Recursive filtering is equivalent but much more efficient, using only the previous belief state and the new evidence.
- Both give the same result; recursive computation exploits the Markov assumption.

10. Key Takeaways

- Filtering estimates the current state.
- Belief state is $P(X_t | e_{1:t})$.
- Use prediction then correction.
- Recursive update depends only on previous belief and new evidence.
- Same idea as forward algorithm.
- Enables real-time tracking and monitoring.
- Fundamental tool in probabilistic AI.

“Filtering allows an agent to maintain a belief about the current state using all past evidence.”

1. What is Smoothing?

- Smoothing is inference about a past state X_t given both past and future evidence (all observations).
- Unlike filtering (uses evidence up to t) or prediction (uses evidence up to t and predicts $t+k$), smoothing uses the entire sequence of observations.
- Gives the probability distribution over a past state, after seeing the whole sequence.

Key Idea: Smoothing uses future evidence to refine our belief about past states.

2. Forward and Backward Messages

- Forward message (α_t): summarizes evidence from the past (up to time t).

$$\alpha_t(X_t) = P(X_t | e_{1:t})$$

(filtering distribution)

- Backward message (β_t): summarizes evidence from the future ($t+1$ to T).

$$\beta_t(X_t) = P(e_{t+1:T} | X_t)$$

- Smoothing distribution combines both:

$$P(X_t | e_{1:T}) = \frac{\alpha_t(X_t) \beta_t(X_t)}{\sum_X \alpha_t(X) \beta_t(X)}$$

α_t incorporates evidence up to t .
 β_t incorporates evidence after t .

3. Forward-Backward Algorithm

Algorithm (Smoothing)

- Forward pass: compute and store $\alpha_t(X_t)$ for $t=1, 2, \dots, T$ using the filtering update (as in forward algorithm).
- Backward pass: compute $\beta_t(X_t)$ for $t=T, T-1, \dots, 1$.
- Combine: for each t , compute $P(X_t | e_{1:T}) \propto \alpha_t(X_t) \beta_t(X_t)$.

Forward message (filtering update):

$$\alpha_t(X_t) = P(e_t | X_t) \sum_{X_{t-1}} P(X_t | X_{t-1}) \alpha_{t-1}(X_{t-1})$$

Backward message:

$$\beta_t(X_t) = \sum_{X_{t+1}} P(e_{t+1} | X_{t+1}) P(X_{t+1} | X_t) \beta_{t+1}(X_{t+1})$$

Initialization:

- $\alpha_1(x_0)$ from initial distribution.
- $\beta_T(x_T) = 1$ for all x_T .

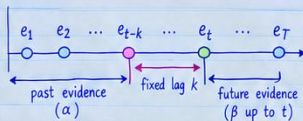
4. Fixed-Lag Smoothing

- In many applications we do not need to wait for all future evidence.
- Fixed-lag smoothing estimates the state X_{t-k} using evidence up to time t (a lag of k time steps).
- Useful for online systems with limited delay and memory.

Formula:

$$P(X_{t-k} | e_{1:t}) \propto \alpha_{t-k}(X_{t-k}) \beta_{t-k}^t(X_{t-k})$$

where $\beta_{t-k}^t(X_{t-k}) = P(e_{t-k+1:t} | X_{t-k})$

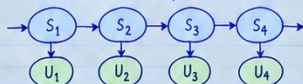


5. Filtering vs Prediction vs Smoothing

Aspect	Prediction	Filtering	Smoothing
Query	$P(X_{t+k} e_{1:t})$	$P(X_t e_{1:t})$	$P(X_t e_{1:T})$
Uses evidence	Past	Past (up to t)	Past and future (all)
Goal	Predict future state	Estimate current state	Infer past state
When used	Planning, control	Online state estimation	Post-processing, better accuracy
Accuracy	Least accurate (more uncertain)	Better	Most accurate (uses more evidence)

6. Example: Umbrella World (Smoothing)

- Hidden state: S_t (weather) $\in$ {Rainy (R), Sunny (S)}
- Observation: U_t (umbrella) $\in$ {Yes, No}
- We observe a sequence of umbrella readings and want to estimate past weather states.



Given observations:

$U_1 = \text{Yes}$ $U_2 = \text{Yes}$
 $U_3 = \text{No}$ $U_4 = \text{No}$

Smoothed belief (example):

$$P(S_t = \text{Rainy} | e_{1:t})$$

t	1	2	3	4
Rainy	0.73	0.62	0.28	0.18
Sunny	0.27	0.38	0.72	0.82

Even though $U_3 = \text{No}$, smoothing still gives some probability of Rainy at $t=3$ because of evidence at $t=1, 2$ and $t=4$.

7. Backward Pass Intuition

- The backward message $\beta_t(x_t)$ measures how likely the future evidence is, given the current state.
- Computed from $t=T$ down to 1 using the same transition and sensor models.

Equation (repeated):

$$\beta_t(X_t) = \sum_{X_{t+1}} P(e_{t+1} | X_{t+1}) P(X_{t+1} | X_t) \beta_{t+1}(X_{t+1})$$

$$\beta_T(x_T) = 1 \quad \text{for all } x_T$$

Intuition:

- $\beta_t(x_t)$ is high if state x_t leads to future observations that are likely.
- It propagates information from the future back to the past.
- Combines with $\alpha_t(x_t)$ to give smoothed estimate.

8. Computational Notes

- Forward and backward passes each take $O(T|X|^2)$ time and $O(T|X|)$ space (to store messages).
- Smoothing requires storing all $\alpha_t(x_t)$ (unlike filtering, which needs only the current α_t).
- For fixed-lag smoothing, we only need to store a limited window of messages.
- Same complexity as filtering, just with an additional backward pass.

9. Applications

- Robot localization (better estimate of past locations after seeing full data).
- Tracking objects in video (e.g. target tracking).
- Speech recognition (infer past words using entire utterance).
- Medical monitoring (infer past hidden conditions from full sequence of tests).
- Natural language processing (HMMs for part-of-speech tagging, smoothing gives better tag estimates).

10. Key Takeaways

- Smoothing estimates a past state using both past and future evidence.
- Combine forward and backward messages: $P(X_t | e_{1:T}) \propto \alpha_t(X_t) \beta_t(X_t)$.
- Forward pass uses evidence up to t .
- Backward pass uses evidence after t .
- Fixed-lag smoothing is useful for online systems.
- Smoothing is more accurate than filtering or prediction.
- Same basic model and complexity as filtering, with an additional backward pass.

1. Most Likely Sequence Problem

- Given a sequence of observations $e_{1:T} = e_1, e_2, \dots, e_T$, find the most likely sequence of hidden states $x_{1:T} = x_1, x_2, \dots, x_T$.
- This is called decoding or the most likely explanation over time.
- We want the state sequence that maximizes the joint probability:

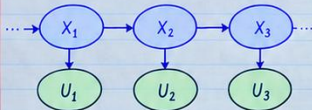
$$x_{1:T}^* = \arg \max_{x_{1:T}} P(x_{1:T}, e_{1:T})$$

$$= \arg \max_{x_{1:T}} P(x_1) \prod_{t=2}^T P(x_t | x_{t-1}) \prod_{t=1}^T P(e_t | x_t)$$

Key Idea: Find the single most likely sequence of states that could have generated the observations.

5. Example: Weather HMM

- Hidden states: weather $X_t \in \{\text{Rain, Sunny}\}$.
- Observations: umbrella $U_t \in \{\text{Yes, No}\}$.
- Given a sequence of umbrella observations, find the most likely sequence of weather states using the Viterbi algorithm.



Model parameters:

Initial distribution $P(X_1)$:		Transition model $P(X_t X_{t-1})$:	
State	$P(X_1)$	From \ To	
Rain	0.6	Rain	0.7
Sunny	0.4	Sunny	0.4

Sensor model $P(U_t | X_t)$:

State	$P(U = \text{Yes})$	$P(U = \text{No})$
Rain	0.9	0.1
Sunny	0.2	0.8

Observation sequence:

$U_1 = \text{Yes}, U_2 = \text{No}, U_3 = \text{Yes}$

Find the most likely weather sequence $X_{1:3}$.

2. Viterbi Idea

- Use dynamic programming to efficiently find the most likely sequence.
- Similar to forward algorithm, but instead of summing over all paths, we keep only the most likely path to each state at each time step.
- Build up solutions for prefixes of the sequence and use optimal substructure.
- Keep track of backpointers to reconstruct the best sequence at the end.

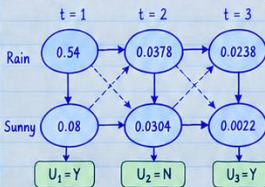
Viterbi variable:

$$\delta_t(x) = \max_{x_{1:t-1}} P(x_{1:t-1}, x_t = x, e_{1:t})$$

= probability of the most likely path that ends in state x at time t , including the observations.

6. Viterbi Trellis for Example

- Trellis shows $\delta_t(x)$ values and backpointers $\psi_t(x)$ for each state at each time step.



Computation (using the model above):

- Initialization ($t = 1$):
 - $\delta_1(\text{Rain}) = 0.6 \times 0.9 = 0.54$
 - $\delta_1(\text{Sunny}) = 0.4 \times 0.2 = 0.08$
- Recursion ($t = 2, U_2 = \text{No}$):
 - $\delta_2(\text{Rain}) = 0.1 \times \max(0.54 \times 0.7, 0.08 \times 0.4) = 0.1 \times 0.378 = 0.0378$
 - $\delta_2(\text{Sunny}) = 0.8 \times \max(0.54 \times 0.3, 0.08 \times 0.6) = 0.8 \times 0.114 = 0.0304$
- Recursion ($t = 3, U_3 = \text{Yes}$):
 - $\delta_3(\text{Rain}) = 0.9 \times \max(0.0378 \times 0.7, 0.0304 \times 0.4) = 0.9 \times 0.02646 = 0.0238$
 - $\delta_3(\text{Sunny}) = 0.2 \times \max(0.0378 \times 0.3, 0.0304 \times 0.6) = 0.2 \times 0.01824 = 0.0022$
- Best final state: Rain ($0.0238 > 0.0022$).
- Backtrack: $\psi_3(\text{Rain}) = \text{Rain}, \psi_2(\text{Rain}) = \text{Rain}$.

Most likely sequence: Rain $\rightarrow$ Rain $\rightarrow$ Rain
(given observations: Yes, No, Yes)

3. Viterbi Recurrence

For $t = 1$ (initialization):

$$\delta_1(x) = P(x) P(e_1 | x) \quad \text{for all } x$$

For $t = 2, \dots, T$ (recurrence):

$$\delta_t(x) = P(e_t | x) \max_{x'} [\delta_{t-1}(x') P(x | x')]$$

Termination:

$$P^* = \max_x \delta_T(x) \quad (\text{probability of the best sequence})$$

4. Backpointers

- To recover the best sequence, store backpointers during the recursion:

$$\psi_t(x) = \arg \max_{x'} [\delta_{t-1}(x') P(x | x')]$$

- $\psi_t(x)$ is the best previous state leading to state x at time t .
- After filling the tables, backtrack from $x_T^* = \arg \max_x \delta_T(x)$ to get $x_{T-1}^*, \dots, x_1^*$.

7. Viterbi vs. Filtering

Aspect	Viterbi (Decoding)	Filtering
Goal	Most likely state sequence $x_{1:T}$	Distribution over current state $P(X_t e_{1:t})$
Computation	Max (keep best path)	Sum (marginalize over all paths)
Output	Single best sequence	Belief state (prob. distribution)
Use case	Interpretation, explanation, labeling	Prediction, decision making, monitoring

8. Complexity

- Let n = number of states, T = number of time steps.
- Time complexity: $O(n^2 T)$
- Space complexity: $O(nT)$ to store δ and ψ (or $O(n)$ with careful implementation).
- Same order as forward algorithm, but with max instead of sum.

9. Step-by-Step Viterbi Algorithm

- Initialize $\delta_1(x) = P(x) P(e_1 | x)$ for all x .
- For $t = 2$ to T :
 - For each state x , compute $\delta_t(x)$ and $\psi_t(x)$.
 - Keep only the best previous state.
- Terminate: find $x_T^* = \arg \max_x \delta_T(x)$.
- Backtrack using ψ to recover $x_{T-1}^*, \dots, x_1^*$.
- Return the most likely sequence and its probability $P^* = \max_x \delta_T(x)$.

10. Applications and Key Takeaways

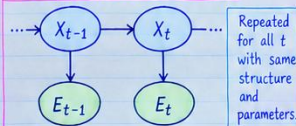
Applications:	Key Takeaways:
- Speech recognition (word sequences) - Part-of-speech tagging (NLP) - Robot localization and mapping - Activity recognition (sensor data) - Bioinformatics (gene finding) - Plan recognition and diagnosis	- Decoding finds the most likely state sequence. - Viterbi uses dynamic programming. - Keep backpointers to recover the sequence. - Same time/space complexity as forward algorithm. - Widely used in many real-world AI problems.

"Viterbi finds the single most likely explanation over time by combining probabilities with dynamic programming."

1. Dynamic Bayesian Networks Recap

- A DBN is a Bayesian network unrolled over time with repeated structure.
- Each time slice contains a set of state variables X_t and observation variables E_t .
- Edges encode the transition model $P(X_t | X_{t-1})$ and the sensor model $P(E_t | X_t)$.
- Assume first-order Markov property and time-invariant model (usually).

DBN structure (two time slices):



4. Exact vs. Approximate Temporal Inference

Aspect	Exact Inference	Approximate Inference (e.g. Particle Filtering)
Method	Forward algorithm, variable elimination	Sampling (Monte Carlo)
State space	Small, discrete	Large, continuous or discrete
Complexity	Exponential in $ X $	Linear in number of particles N
Result	Exact belief state $P(X_t e_{1:t})$	Approximate belief (using samples)
When to use	When state space is small and finite	When state space is large, continuous or intractable
Pros	Exact, no sampling error	Scalable, model nonlinear/continuous
Cons	Intractable for large spaces	Sampling error, needs many particles

7. Summary of Temporal Reasoning Tasks

Task	Goal	Typical Method(s)
Filtering (monitoring)	Compute current belief $P(X_t e_{1:t})$	Forward algorithm, particle filtering
Prediction	Compute future belief $P(X_{t+k} e_{1:t})$	Iterated prediction (using transition model)
Smoothing	Compute past belief $P(X_t e_{1:T})$	Forward-backward algorithm
Most likely explanation	Find most likely state sequence $x_{1:T}$ given $e_{1:T}$	Viterbi algorithm (HMMs)
Learning	Learn model parameters	EM algorithm (for HMMs/DBNs)

2. Approximate Inference over Time

- Exact inference (e.g. forward algorithm) can be infeasible when the state space is large or continuous.
- We can use approximate methods such as particle filtering (Monte Carlo) to maintain a set of samples (particles) representing the belief state.
- Useful for large, continuous or nonlinear state spaces where exact inference is intractable.
- Trade-off: approximate methods are scalable but introduce sampling error.

Key Idea: Use a set of weighted samples to represent the belief state and update them over time using the transition and evidence models.

3. Particle Filtering (Sequential Monte Carlo)

- Maintains a set of weighted particles $\{(x_t^{[i]}, w_t^{[i]})\}_{i=1}^N$ to represent $P(X_t | e_{1:t})$.
- Each step: predict (sample), weight (using evidence), and resample.
- Works well for large or continuous state spaces (e.g. robot localization).

Particle filtering algorithm (outline)

- Initialize N particles $x_0^{[i]} \sim P(X_0)$, set weights $w_0^{[i]} = 1/N$.
- For each time step $t = 1, 2, \dots$
 - Sample $x_t^{[i]} \sim P(x_t | x_{t-1}^{[i]})$ (prediction)
 - Compute weight $w_t^{[i]} = P(e_t | x_t^{[i]})$
 - Normalize weights so $\sum_i w_t^{[i]} = 1$
 - Resample N particles proportional to $w_t^{[i]}$ (to avoid degeneracy).

5. Applications in AI

- Robot localization (tracking a moving robot).
- Tracking multiple objects (e.g. SLAM, multiple target tracking).
- Medical monitoring (tracking patient state).
- Weather tracking (e.g. umbrella world).
- Speech and gesture recognition (hidden state models).
- Economics and finance (tracking hidden market variables).
- Fault diagnosis and monitoring (e.g. in engineering systems).

Example: Robot localization

- State X_t : robot's location (continuous).
- Action: move with noisy motion model.
- Observation E_t : noisy sensor reading (e.g. sonar).
- Use particle filtering to maintain belief over possible locations.

6. Key Formulas Recap

Filtering (forward) step:

$$P(X_t | e_{1:t}) = \frac{\alpha P(e_t | X_t)}{\sum_{x_{t-1}} P(X_t | x_{t-1}) P(x_{t-1} | e_{1:t-1})}$$

α : normalization constant.

Prediction (no evidence):

$$P(X_t | e_{1:t-1}) = \sum_{x_{t-1}} P(X_t | x_{t-1}) P(x_{t-1} | e_{1:t-1})$$

Particle weight update:

$$w_t^{[i]} = P(e_t | x_t^{[i]})$$

(often multiplied with previous weight before normalization)

Normalize:

$$w_t^{[i]} \leftarrow \frac{w_t^{[i]}}{\sum_{j=1}^N w_t^{[j]}}$$

8. Common Exam Questions

- Derive the filtering equation for a DBN.
- Explain the particle filtering algorithm and why resampling is needed.
- Compare exact inference and particle filtering. When would you use each?
- Given a simple HMM/DBN, compute $P(X_t | e_{1:t})$ for a few time steps.
- Apply the Viterbi algorithm to find the most likely state sequence.
- Explain the difference between filtering, prediction, and smoothing.
- Discuss the Markov assumption in DBNs.
- How would you handle a large or continuous state space?

9. Common Mistakes

- Forgetting to normalize particle weights.
- Using too few particles (high variance).
- Confusing filtering with prediction or smoothing.
- Ignoring the Markov assumption.
- Not resampling (leads to particle degeneracy).
- Using an incorrect sensor or transition model.
- Assuming exact inference always feasible.
- Confusing Viterbi (most likely sequence) with filtering (belief state).

10. Final Key Takeaways

- DBNs model sequential domains with repeated structure.
- Filtering, prediction, smoothing and most likely explanation are key inference tasks.
- Exact inference is feasible for small discrete state spaces; otherwise use approximate methods.
- Particle filtering provides a simple and scalable way to approximate filtering.
- Probabilistic temporal models are widely used in real-world AI applications.

“Probabilistic models over time let an agent maintain and update its beliefs in a changing, uncertain world.”

— AIMA (Ch.15)